Introduction to Serverless Computing with AWS Lambda

Simplifying Cloud Application Development Without Managing Servers

by: Mohamad Ikhsan Nurulloh

What is Serverless Computing?

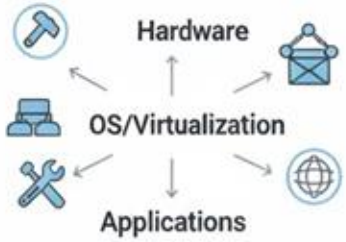
Serverless computing lets developers run code without managing servers. The cloud provider handles scaling and maintenance, and you only pay when your code runs, making it efficient, fast, and scalable.

Serverless ≠ No Server — servers exist but are managed for you.
Function as a Service (FaaS) — e.g., **AWS Lambda**

On Prem
vs
Traditional Cloud
vs
Serverless

On-Premises Computing

You Manage Everything:



Expensive
Infrastrcture Investment.
High Maintenance.

Traditional Cloud

You Manage:
OS/VMs, Containers,
Data, Apps.

Less Management Overhead.
More Flexibility

Cloud Provider Manages:
Hardware, Data Center,
Basic Infrastucture

Rent Infrastucture.
Scale Manually.

Serverless

Cloud Provider Manages
All Infrastucture

You Focus On:
Code & Business Logic

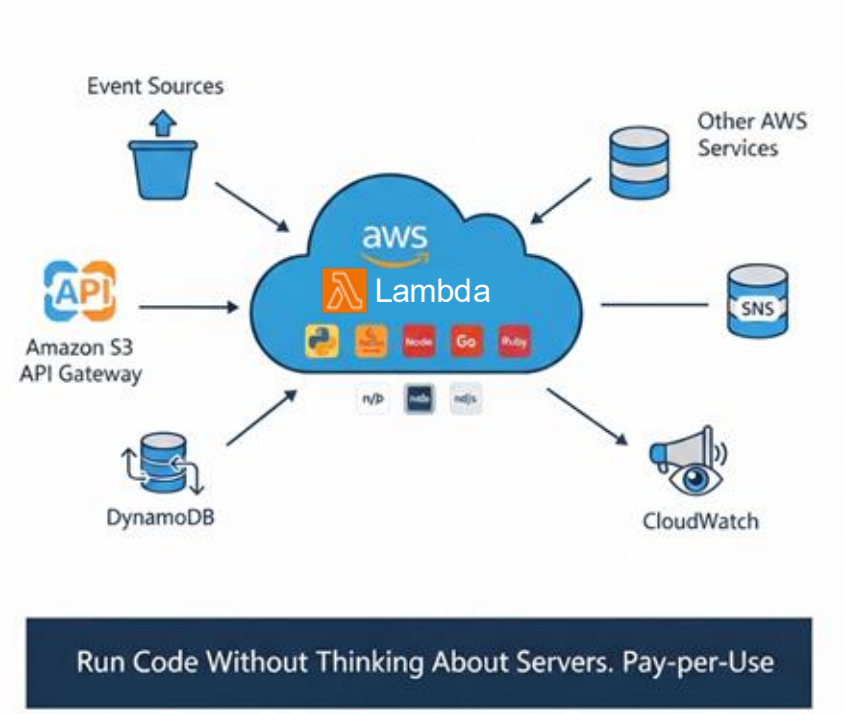
No Servers to Manage.

Pay-per-Execution.
Automatic Scaling.

Introducing AWS Lambda

AWS Lambda is Amazon's FaaS offering.

- You write a function → upload → AWS runs it automatically.
- Supports multiple languages: Python, Node.js, Java, Go, C#, etc.
- Triggered by events from AWS or external sources.
- A serverless computing service that runs code without provisioning or managing servers.



AWS Lambda Free Tier

- 1,000,000 free requests per month
- 400,000 GB-seconds of compute time per month

Introducing AWS Lambda

AWS Lambda is Amazon's FaaS offering.

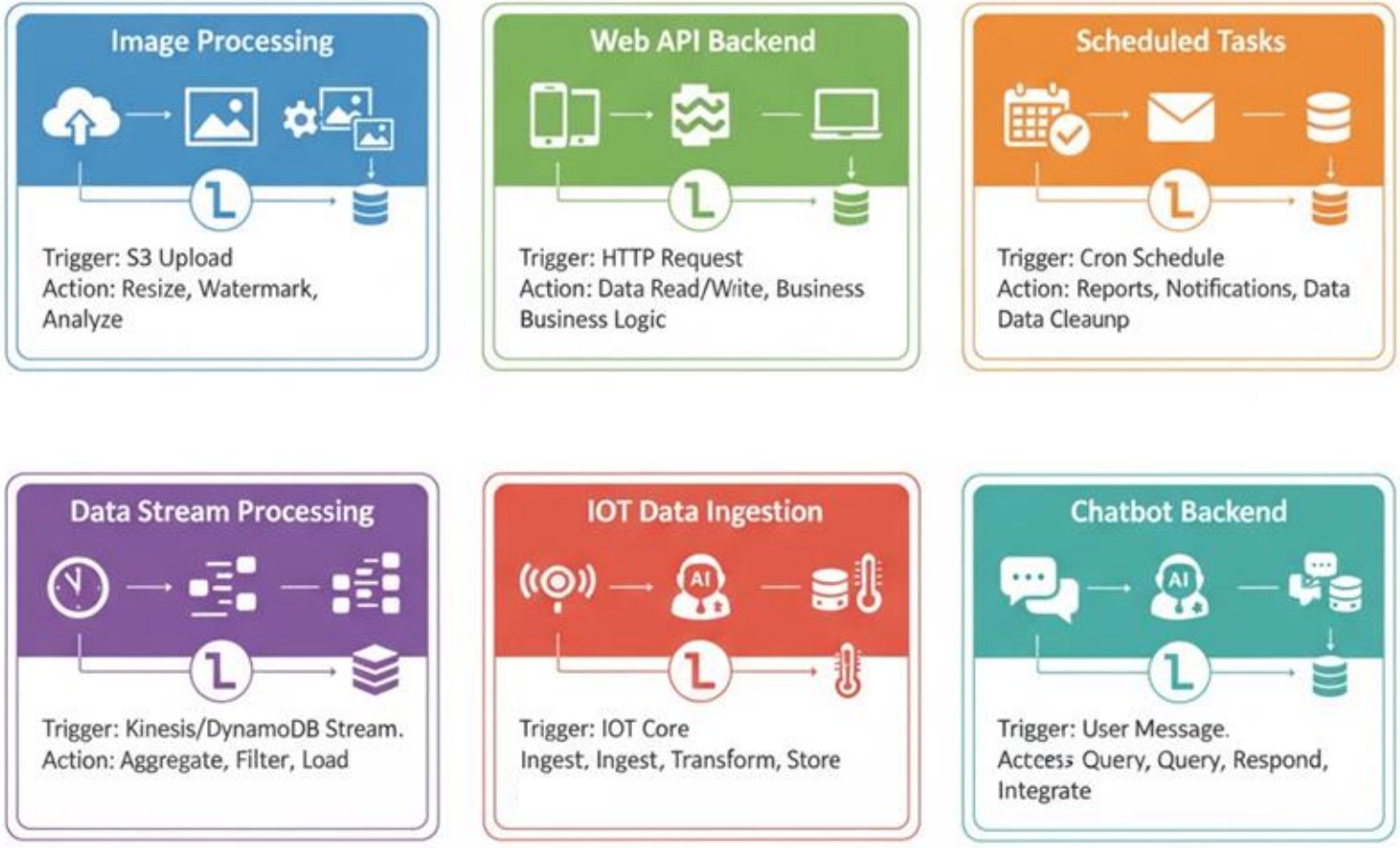
- It executes code in response to events such as changes in data, HTTP requests, or system triggers.
- Automatically handles infrastructure tasks like scaling, patching, and monitoring.
- Charges only for the compute time consumed while the code runs, making it cost-efficient and scalable.

AWS Lambda Limitations

- Maximum execution time: 15 minutes
- Memory limits: 128 MB to 10 GB (CPU scales with memory)
- Local temporary storage only: /tmp up to 10 GB (non-persistent)
- Deployment package size limits (ZIP up to 50 MB, Container up to 10 GB)

How AWS Lambda Works

Example Use Cases



Suitable For / Not Suitable For

Where Lambda Excels

Reactive Automation

Instantly responds; 'if this, then that' logic.

Quick Bursts of Work

Speedy executions under15 mintes, not maratoners.

Elastic Scaling

Handles massive, unpexidiable traffic spikes instantly & automatically.

Where Lambda Falls Short

Endurance Tasks

Not for continuous processing long-ruming compuations

Stateful Communication

No persistent connections for live chat chat or gaming servers

Monlothic Architectures

Traditional, tightly-coupled app are a poor fit

Simple Lambda Example (Python)

```python
def lambda_handler(event, context):
    """Simple AWS Lambda that returns a greeting message."""

    # You can pass a 'name' key via the test event
    name = event.get('name', 'World')

    message = f"Hello, {name}! This is AWS Lambda running in Python."

    return {
        'statusCode': 200,
        'body': message
    }
```

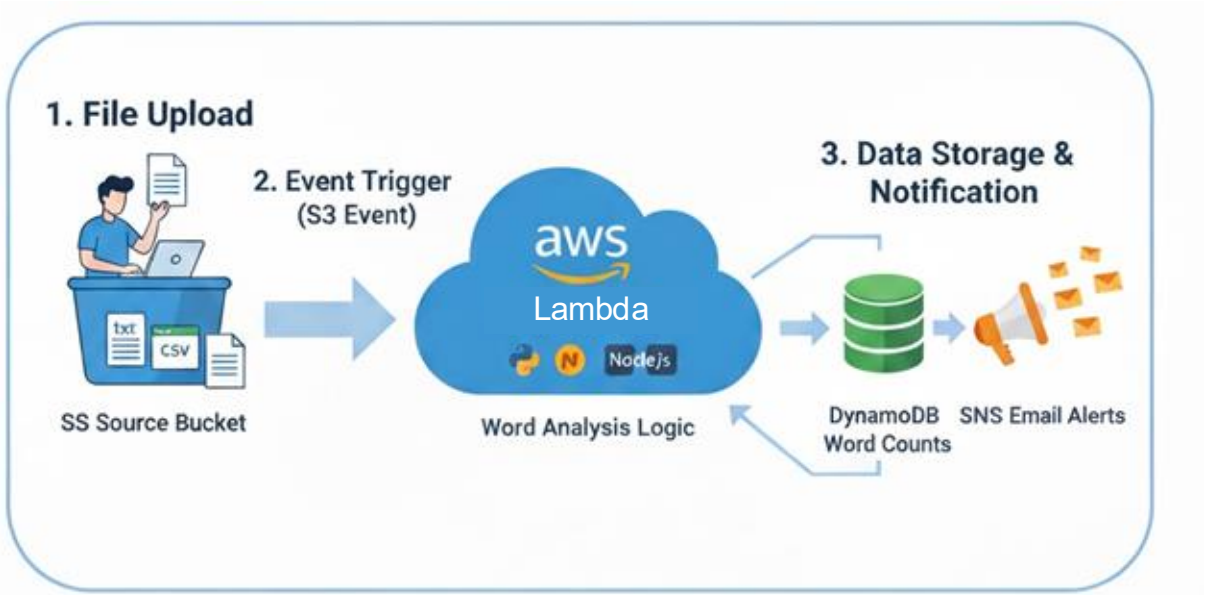
Deploy via AWS Console or CLI
Test with event: { "name": "Ikhsan" }

Business Case: XYZ Company

Current Challenge:

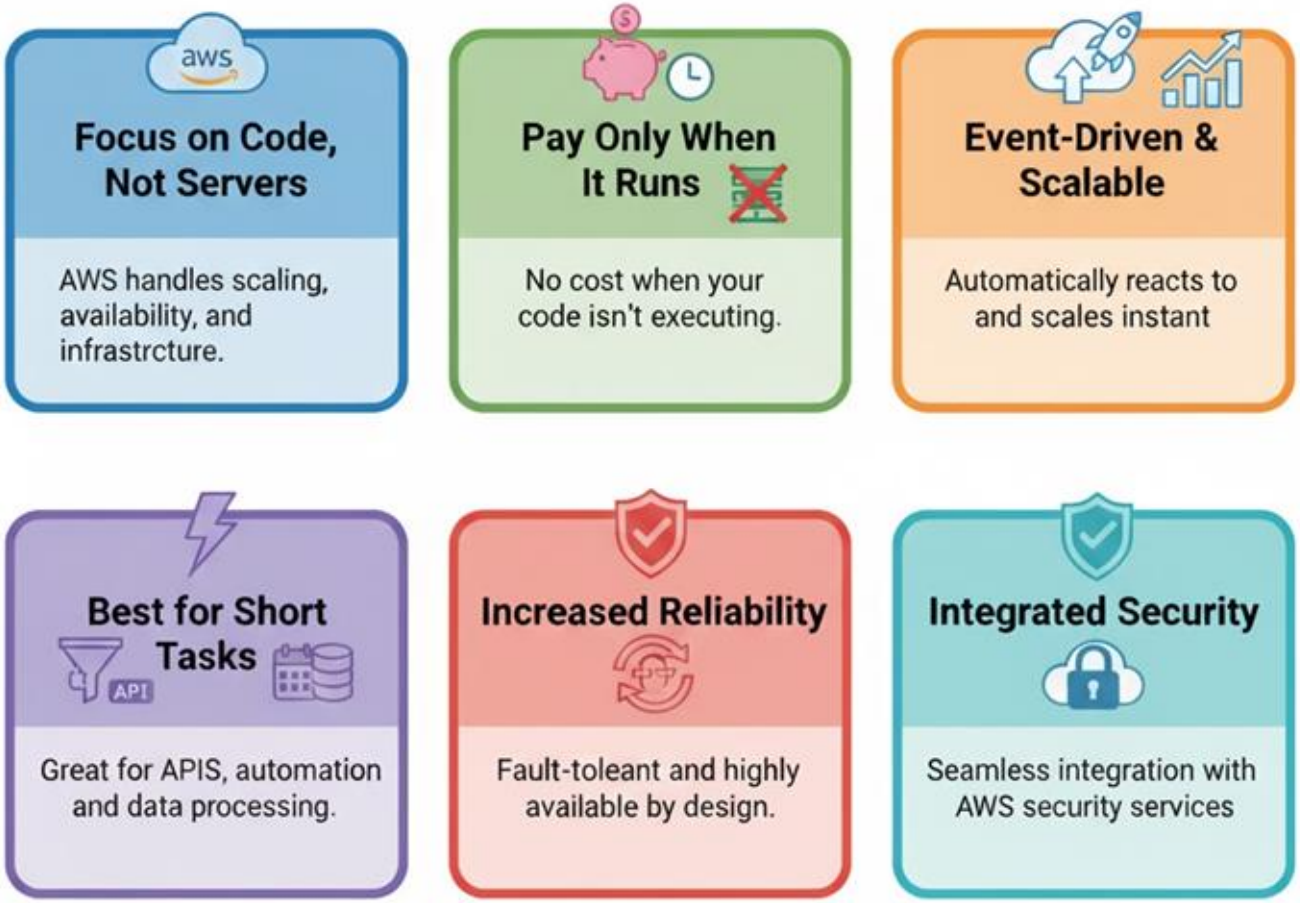
XYZ Company operates in corporate document and compliance processing, where it routinely receives client documents in PDF and text formats (**1–2 MB each**). These files arrive at **unpredictable times** and are currently manually **saved, reviewed, extracted for key information**, and then communicated to stakeholders via **email**. Each file takes **1–5 minutes to handle**. The company prefers **not to manage any server** infrastructure and wants to **automate** this entire process to improve efficiency and reliability.

Solution



https://github.com/ikhsannur1996/aws-serverless

Key Takeaways



Thank You

linkedin.com/in/mohamad-ikhsan-nurulloh/

github.com/ikhsannur1996

Reference

https://aws.amazon.com/serverless/

https://aws.amazon.com/tutorials/run-serverless-code/

https://aws.amazon.com/lambda/

https://docs.aws.amazon.com/lambda/latest/dg/getting-started.html